

Báo cáo bài thực hành Caro AI

1. Mô tả bài toán

Chương trình xây dựng game cờ Caro giữa người chơi và máy tính trên bàn cờ 15x15. Máy tính chọn nước đi bằng thuật toán tìm kiếm đối kháng Minimax và phiên bản cải tiến Alpha-Beta pruning.

2. Luật chơi và điều kiện thắng

Hai bên đánh luân phiên vào các ô trống. Quân đen đi trước. Một bên thắng khi có 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc một trong hai đường chéo. Không xét luật chặn hai đầu. Nếu bàn cờ đầy mà không có người thắng thì hòa.

3. Biểu diễn trạng thái bàn cờ

Bàn cờ được biểu diễn bằng mảng hai chiều boardMap[15][15]:

- "0": ô trống.
- "1": quân đen, cũng là người chơi MAX trong hàm đánh giá.
- "-1": quân trắng, cũng là người chơi MIN trong hàm đánh giá.

Các thông tin phụ gồm currentI, currentJ, lastPlayed, emptyCells, boardValue, nextBound, nodes_visited và rollingHash.

4. Sinh nước đi hợp lệ

Hàm isValid(i, j) kiểm tra tọa độ nằm trong bàn cờ và ô chưa có quân. Để giảm không gian tìm kiếm, chương trình không duyệt toàn bộ bàn sau khi đã có quân, mà chỉ sinh các ô trống nằm gần quân đã đánh trong nextBound. Danh sách nước đi được sắp xếp theo điểm heuristic và độ gần trung tâm.

5. Kiểm tra trạng thái kết thúc

Sau mỗi nước đi, chương trình kiểm tra 4 hướng: ngang, dọc, chéo chính và chéo phụ. Nếu đếm được ít nhất 4 quân liên tiếp cùng màu thì trả về người thắng. Nếu không còn ô trống thì trả về hòa.

6. Thuật toán Minimax

Minimax được cài đặt trong source/AI.py. Đây là thuật toán tìm kiếm đối kháng cho trò chơi hai người tổng bằng không. Chương trình quy ước quân đen 1 là MAX và quân trắng -1 là MIN. MAX cố gắng làm điểm đánh giá lớn nhất, còn MIN cố gắng làm điểm đánh giá nhỏ nhất.

Quy trình xử lý của Minimax trong chương trình:

- Nếu trạng thái hiện tại đã kết thúc, hàm trả về điểm kết thúc: thắng của MAX là điểm dương rất lớn, thắng của MIN là điểm âm rất lớn, hòa là 0.
- Nếu chưa kết thúc nhưng đã đạt giới hạn độ sâu, hàm trả về điểm heuristic của bàn cờ hiện tại.
- Nếu là lượt MAX, chương trình thử từng nước đi hợp lệ, gọi đệ quy cho lượt MIN, rồi chọn giá trị lớn nhất.
- Nếu là lượt MIN, chương trình thử từng nước đi hợp lệ, gọi đệ quy cho lượt MAX, rồi chọn giá trị nhỏ nhất.
- Ở tầng gốc của cây tìm kiếm, chương trình lưu lại nước đi tốt nhất, giá trị đánh giá, số node đã xét và thời gian chạy.

Vì bàn cờ 15x15 có rất nhiều ô trống, chương trình không sinh toàn bộ nước đi trên bàn sau khi ván đã bắt đầu. Thay vào đó, childNodes() chỉ xét các ô trống nằm gần những quân đã đánh trong nextBound. Cách này vẫn giữ được các nước đi có ý nghĩa chiến thuật trong đa số tình huống, đồng thời giảm hệ số phân nhánh.

7. Thuật toán Alpha-Beta

Alpha-Beta pruning là cải tiến của Minimax, dùng cùng trạng thái, cùng hàm đánh giá và cùng độ sâu tìm kiếm. Kết quả lý thuyết của Alpha-Beta phải giống Minimax nếu thứ tự duyệt nước đi và hàm đánh giá không đổi; điểm khác biệt là Alpha-Beta có thể bỏ qua các nhánh không còn khả năng ảnh hưởng đến lựa chọn cuối cùng.

Hai biến chính:

- alpha: giá trị tốt nhất mà MAX hiện có thể đảm bảo trên đường đi đang xét.
- beta: giá trị tốt nhất mà MIN hiện có thể đảm bảo trên đường đi đang xét.

Khi đang duyệt một node MAX, nếu tìm được giá trị tốt hơn thì cập nhật alpha. Khi đang duyệt một node MIN, nếu tìm được giá trị nhỏ hơn thì cập nhật beta. Nếu tại một thời điểm $\beta \leq \alpha$, nhánh còn lại không cần xét tiếp vì đối thủ đã có lựa chọn tốt hơn ở node cha. Điều này không làm thay đổi nước đi tối ưu nhưng giảm đáng kể số trạng thái phải mở rộng.

Trong chương trình, Alpha-Beta còn kết hợp sắp xếp nước đi trước khi duyệt: các ô có điểm heuristic cao và gần trung tâm được xét trước. Thứ tự tốt giúp điều kiện cắt nhánh xuất hiện sớm hơn. Chương trình cũng có bảng chuyển vị đơn giản dùng Zobrist hashing để tái sử dụng giá trị đã tìm trong cùng lượt AI.

8. Hàm đánh giá trạng thái

Hàm đánh giá tính điểm từ góc nhìn quân đen (1). Các mẫu quan trọng:

- 4 quân liên tiếp: điểm rất lớn.
- 3 quân mở hoặc bị chặn một đầu: điểm cao.
- 3 quân có lỗ hổng: điểm trung bình cao.
- 2 quân mở: điểm nhỏ.

Điểm của quân trắng là điểm âm tương ứng, giúp MIN chọn nước làm giảm lợi thế của MAX.

9. Thiết kế trạng thái thử nghiệm

File `performance_eval.py` định nghĩa 5 trạng thái:

- Đầu ván.
- Giữa ván.
- AI có thể thắng ngay.
- Người chơi sắp thắng, AI cần chặn.
- Hai bên cùng có cơ hội tấn công.

Mỗi trạng thái được chạy với depth 1, 2 và 3.

10. Bảng kết quả thực nghiệm

Kết quả đầy đủ nằm trong `ket_qua_thuc_nghiem.csv`. Bảng tóm tắt tại depth 3:

Trạng thái	Minimax move	Minimax nodes	Minimax time	Alpha-Beta move	Alpha-Beta nodes	Alpha-Beta time	Giảm node
Đầu ván	(7,7)	98	0.039452s	(7,7)	28	0.011692s	71.43%
Giữa ván	(6,6)	14263	5.799513s	(6,6)	654	0.260702s	95.41%

AI có thể thắng ngay	(7,9)	10675	4.347338s	(7,9)	459	0.190813s	95.70%
Người sắp thắng cần chặn	(7,9)	10718	4.296740s	(7,9)	113	0.044616s	98.95%
Hai bên cùng tấn công	(10,7)	7707	3.074819s	(10,7)	415	0.170060s	94.62%

- move là nước đi AI chọn, theo dạng (hàng, cột) trong mảng bàn cờ 15x15.
- nodes là số trạng thái thuật toán đã xét trong quá trình tìm kiếm.
- time là thời gian chạy cho một lần chọn nước đi.
- Giảm node được tính theo công thức $1 - \text{alphabet_nodes} / \text{minimax_nodes}$.
- same_move trong file CSV cho biết Alpha-Beta có chọn cùng nước đi với Minimax hay không.

Trong cả 5 trạng thái ở depth 3, Alpha-Beta chọn cùng nước đi với Minimax. Điều này phù hợp với yêu cầu so sánh công bằng: hai thuật toán dùng cùng hàm đánh giá và cùng độ sâu. Khác biệt lớn nhất nằm ở số trạng thái đã xét. Ví dụ ở trạng thái "Người sắp thắng cần chặn", Minimax xét 10718 node, còn Alpha-Beta chỉ xét 113 node, giảm 98.95%.

11. Nhận xét về số trạng thái và thời gian

Ở depth 1, Alpha-Beta gần như không giảm node vì cây tìm kiếm quá nông: thuật toán chỉ cần xét các nước đi trực tiếp, chưa có nhiều cơ hội để cắt nhánh. Vì vậy thời gian của Minimax và Alpha-Beta gần tương đương.

Từ depth 2 trở đi, lợi ích của Alpha-Beta rõ hơn. Khi Minimax phải xét đầy đủ nhiều phản ứng của đối thủ, Alpha-Beta có thể dừng sớm những nhánh mà kết quả chắc chắn không tốt hơn lựa chọn hiện có. Ở depth 3, các trạng thái giữa ván cho thấy mức giảm node thường trên 94%, làm thời gian giảm từ vài giây xuống dưới một giây.

Kết quả cũng cho thấy thứ tự duyệt nước đi có ảnh hưởng lớn. Vì chương trình ưu tiên các nước gần trung tâm và có điểm pattern cao, Alpha-Beta thường gặp nước tốt sớm, từ đó tăng khả năng cắt các nhánh sau.

12. Ảnh hưởng của độ sâu tìm kiếm

Depth càng lớn thì AI nhìn trước được nhiều phản ứng của đối thủ hơn. Ví dụ trạng thái cần chặn ở depth 1 chỉ dựa mạnh vào heuristic, còn depth 2 và 3 đánh giá thêm nước phản công của đối thủ. Đổi lại, số node của Minimax tăng rất nhanh; Alpha-Beta giúp depth 3 vẫn dùng được trong GUI.

13. Ưu điểm và hạn chế

Ưu điểm:

- Chạy được đủ bốn chế độ: Human vs AI, AI vs Human, AI vs AI, Human vs Human.
- Có cả Minimax và Alpha-Beta để so sánh công bằng trên cùng hàm đánh giá.
- Có ghi nhận nước đi, giá trị, depth, số node và thời gian.
- Sinh nước đi gần quân đã đánh nên tốc độ tốt hơn duyệt toàn bàn.

Hạn chế:

- Hàm đánh giá vẫn dựa vào pattern thủ công nên có thể bỏ sót thể cờ phức tạp.

- Chưa có iterative deepening hoặc giới hạn thời gian cứng cho từng nước.
- Chưa phân biệt nhiều cấp độ khó ngoài tham số depth.

14. Link repository GitHub của nhóm

Repository GitHub của nhóm: https://github.com/fyb174147/23020515_23020518_23020525_CaroAI

15. Tài liệu tham khảo

- Repo mẫu <https://github.com/husus/gomokuAI-py>: tham khảo cách tổ chức project, giao diện Pygame, cách biểu diễn bàn cờ bằng mảng hai chiều, ý tưởng gom nhóm nước đi gần quân đã đánh và script đo hiệu năng. Đây là repo mẫu chính được sử dụng trong quá trình làm bài.

- Tham khảo các thuật toán minimax, alpha-beta pruning trên GeeksforGeeks.

- Có sử dụng sự hỗ trợ của các mô hình ngôn ngữ lớn (LLM) trong quá trình sinh mã, sửa lỗi và điều chỉnh mã để phù hợp với cấu trúc repo có sẵn. Nhóm vẫn kiểm tra, chạy thử và điều chỉnh lại logic theo yêu cầu đề bài.

Phần đã điều chỉnh so với repo mẫu: luật thắng 4 quân theo đề, không xét chặn hai đầu, hỗ trợ bốn chế độ chơi, sửa Minimax/Alpha-Beta để hoạt động cho cả quân đen và quân trắng, thêm benchmark 5 trạng thái, lưu kết quả thực nghiệm và viết báo cáo phân tích.